report.md

Мои результаты.

Set up poetry

I use poetry, so I just set it up. Also added linter (basedpyright) and formatter (black + isort).

Задание 1

Results before fix

Изначальные гиперпараметры в main.py:

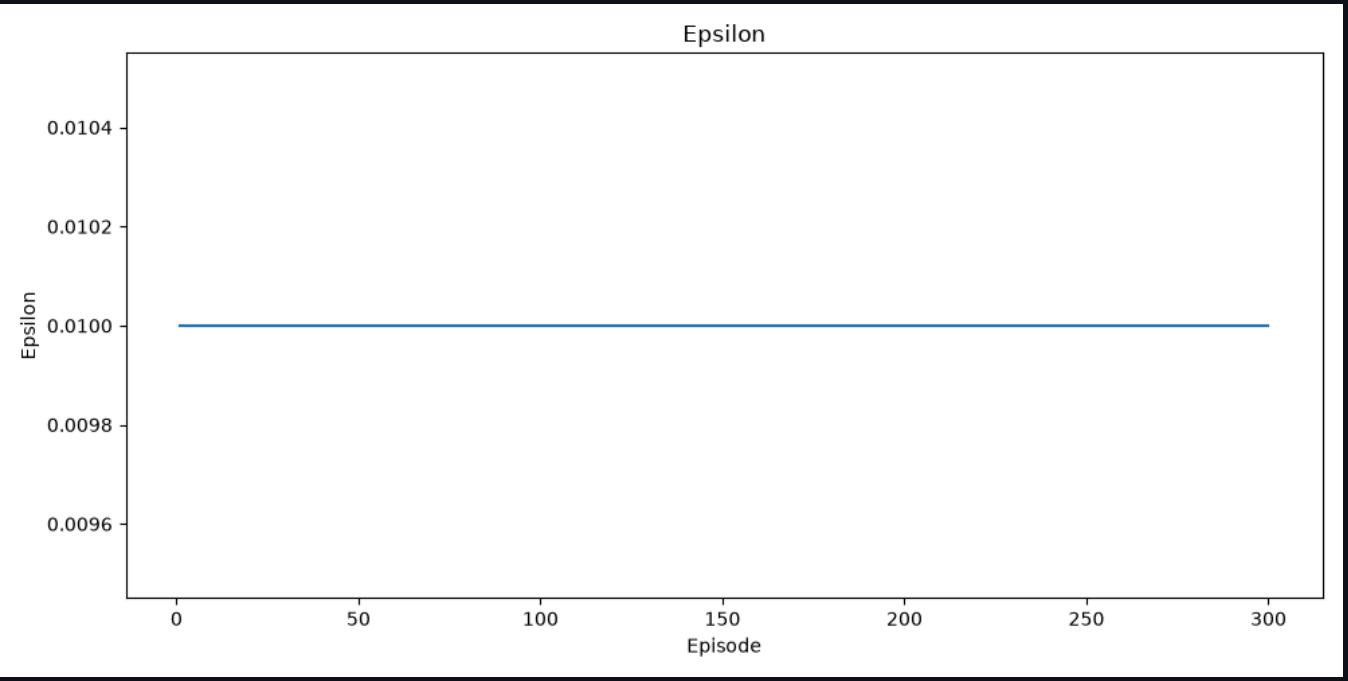
```
gamma=0.90,
learning_rate=1e-2,
batch_size=128,
epsilon_start=0.10,
epsilon_end=0.01,
epsilon_decay=0.995,
target_update_freq=1,
```

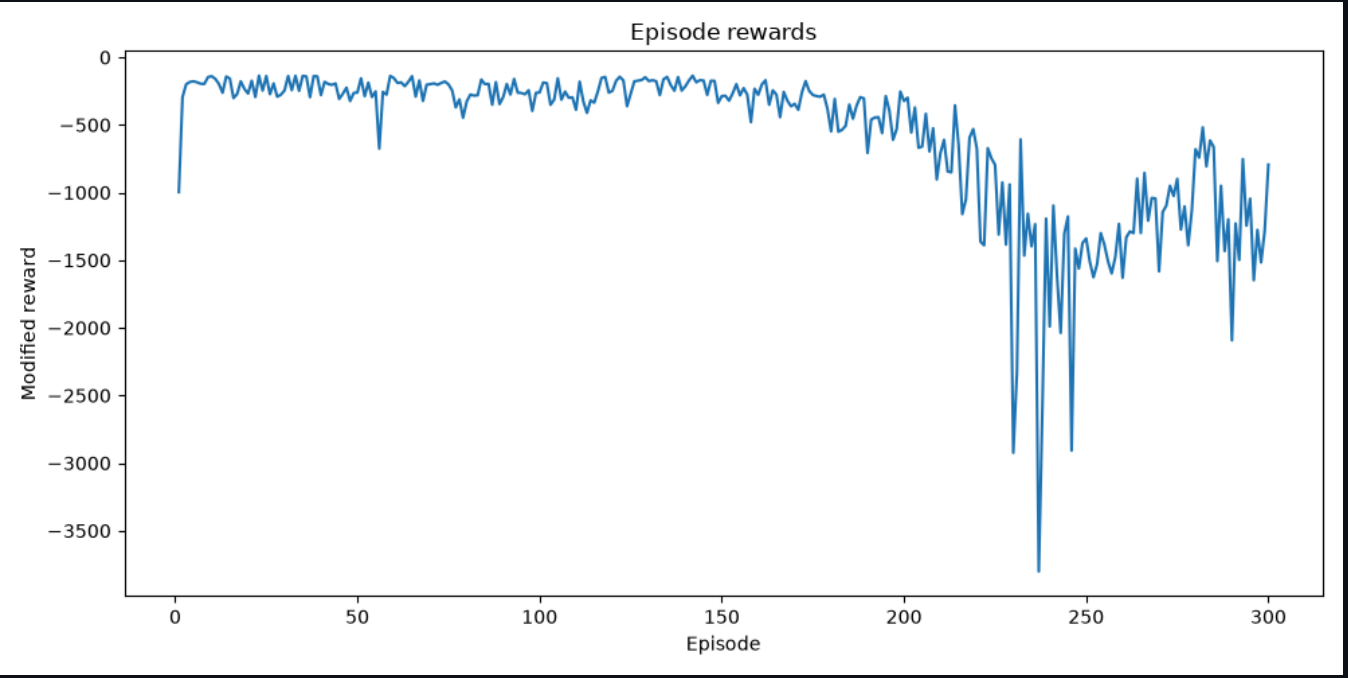
Я запускаю на ноуте на CPU.

```
❱ python -u main.py --mode train | ts -s
00:00:10 Episode   25 | Reward  -138.76 | Epsilon 0.0100
00:00:16 Episode   50 | Reward  -260.60 | Epsilon 0.0100
00:00:23 Episode   75 | Reward  -200.30 | Epsilon 0.0100
00:00:31 Episode  100 | Reward  -260.37 | Epsilon 0.0100
00:00:39 Episode  125 | Reward  -268.54 | Epsilon 0.0100
00:00:45 Episode  150 | Reward  -287.16 | Epsilon 0.0100
00:00:53 Episode  175 | Reward  -279.03 | Epsilon 0.0100
00:01:07 Episode  200 | Reward  -323.36 | Epsilon 0.0100
00:01:30 Episode  225 | Reward  -795.13 | Epsilon 0.0100
00:02:25 Episode  250 | Reward -1339.60 | Epsilon 0.0100
00:03:11 Episode  275 | Reward  -898.71 | Epsilon 0.0100
00:03:53 Episode  300 | Reward  -793.79 | Epsilon 0.0100
```

> I added ts to track time. python -u to avoid stdout buffering.

```
❱ python -u main.py --mode evaluate
{'mean_reward': -200.0, 'std_reward': 0.0, 'min_reward': -200.0,
'max_reward': -200.0, 'successes': 0, 'episodes': 20}
```





Все результаты сохранены в папке `results/results_before-fix/`.

Выводы

- Проблема с epsilon скорее всего из-за плохого выбора epsilon_decay. К 25

шагу epsilon уже epsilon_end. Стоит взять epsilon_decay=0.999.

Epsilon должен меняться от `epsilon_start` до `epsilon_end` уменьшаясь в `epsilon_decay` раз.

```
epsilon_start=0.10,
epsilon_end=0.01,
epsilon_decay=0.995,
```

epsilon влияет на жадность политики. При маленьком epsilon политика довольно жадная и агент не исследует среду.

- loss увеличивается, хотя должен уменьшаться.
- Награды колеблются от -220 до -3500. Награда уменьшается, хотя должна увеличиваться.
- На видео видно, что машинка не понимает, что делает.

Баги

targets

Баг с target. В коде есть ошибка с подсчетом targets: 107 строчка: `targets = rewards + self.gamma * next_q * dones`. Нужно `targets = rewards + self.gamma * next_q * (1 - dones)`.

dones показывает, терминальная траектория или нет. Мы не можем учитывать Q-функцию из терминального состояния, поэтому мы умножаем ее на (1-dones).

Влияние на обучение: ломает обучение.

target_update_freq

Не используется `target_update_freq`. target_net обновляется каждый шаг, что неправильно. Этот баг неявно привел к бесполезности target_net. DQN стал наивнее.

Было:

```
self.target_net.load_state_dict(self.online_net.state_dict())
```

Стало:

```
if self.learn_steps % self.target_update_freq == 0:
    _ = self.target_net.load_state_dict(self.online_net.state_dict())
```

Сейчас в main:

```
target_update_freq=1,
```

Поэтому на обучении это не скажется.

truncated

Мы не используем truncated, которое возвращает env.step.

```
new_state, reward, terminated, truncated, _ = env.step(action)
```

truncated показывает, была ли обрезана траектория по внешним лимитам, типа максимальное кол-во переходов.

Еще важно передать в agent.remember не done, а terminated.

Было:

```
done = terminated

agent.remember(
    state,
    action,
    reward,
    new_state,
    done,
)
```

Стало:

```
done = terminated or truncated

agent.remember(
    state,
    action,
    reward,
    new_state,
    terminated,
)
```

Нет синхронизации `target_net` с `online_net` в `__init__`.

Мы должны инициализировать target_net весами из online_net, иначе до обновления (до target_update_freq) мы будем использовать рандомную сеть.

Идея DQN в том, чтобы target_net следовала за online_net с запозданием.

Было:

```
self.online_net = online_net.to(self.device)
self.target_net = target_net.to(self.device)
```

Стало:

```
self.online_net = online_net.to(self.device)
self.target_net = target_net.to(self.device)

_ = self.target_net.load_state_dict(self.online_net.state_dict())
```

Сейчас в main:

```
target_update_freq=1,
```

Поэтому на обучении это практически не скажется.

seed

seed фиксируется только для env. Стоит фиксировать seed для DQN.

но это не критично.

.train() .eval() inference_mode

нет model.train(), model.eval()

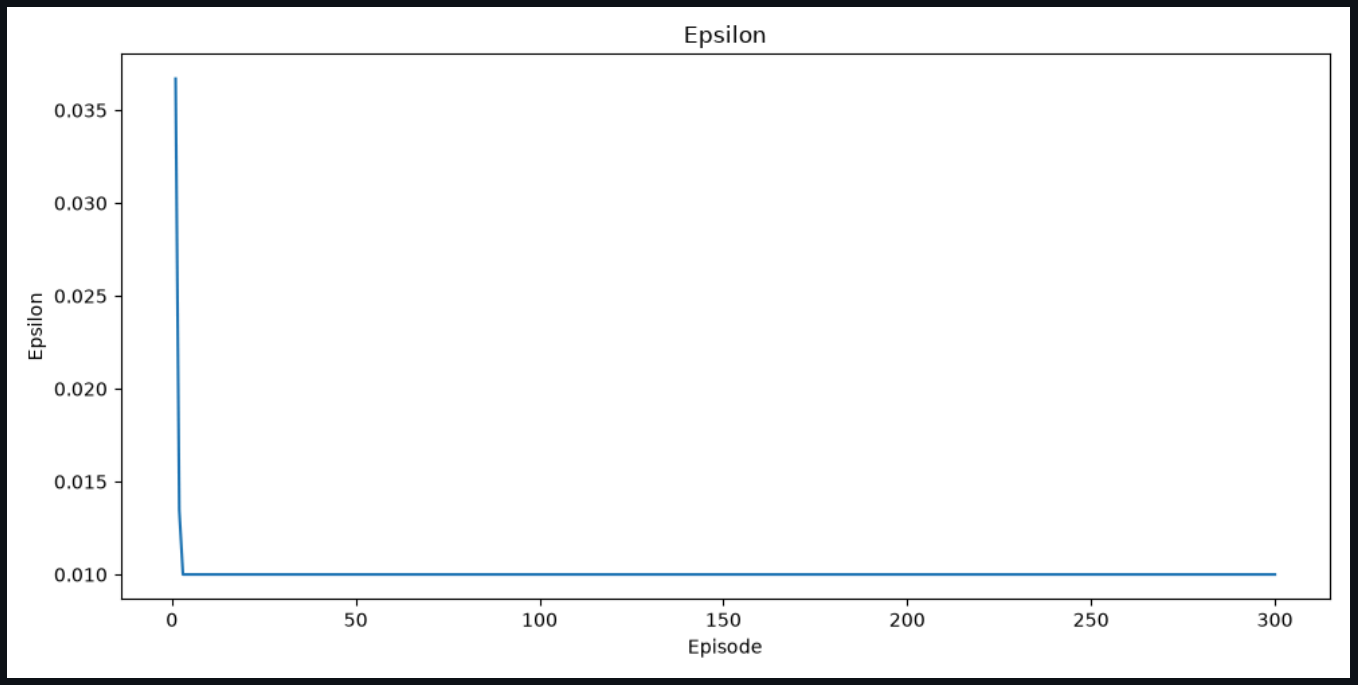
нет @torch.inference_mode() на evaluate.

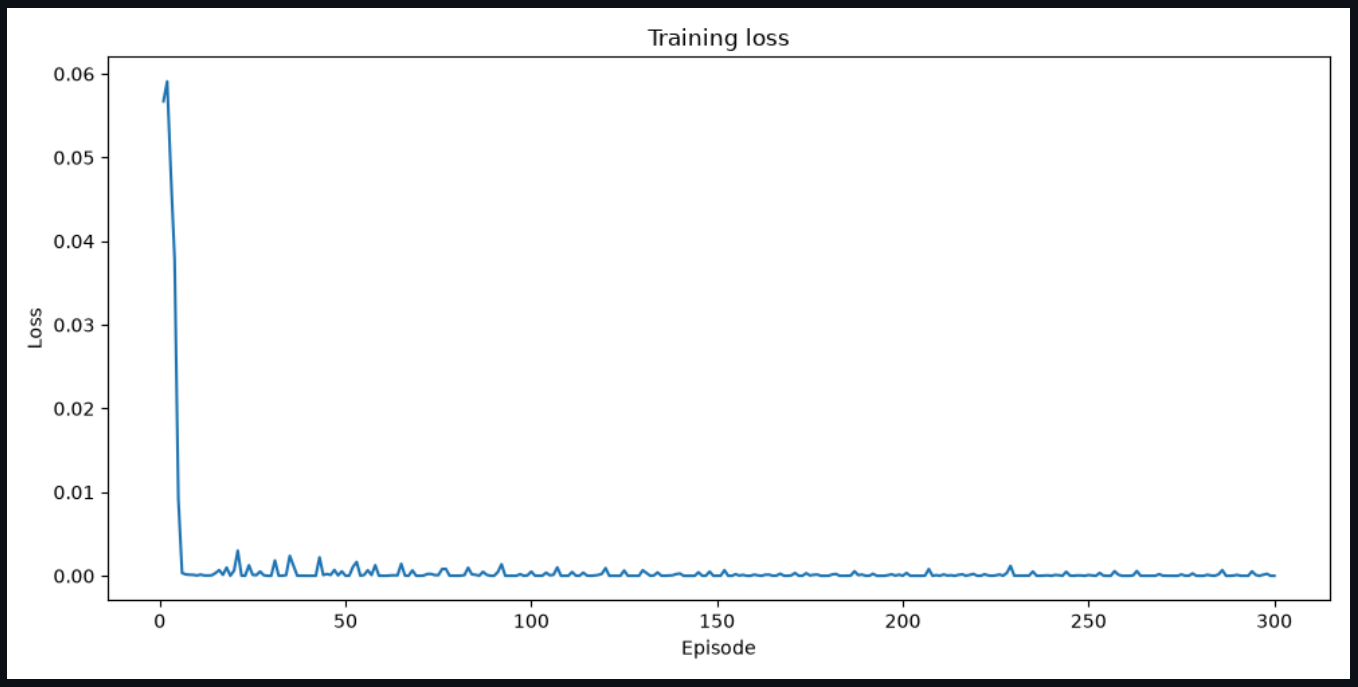
но это не критично.

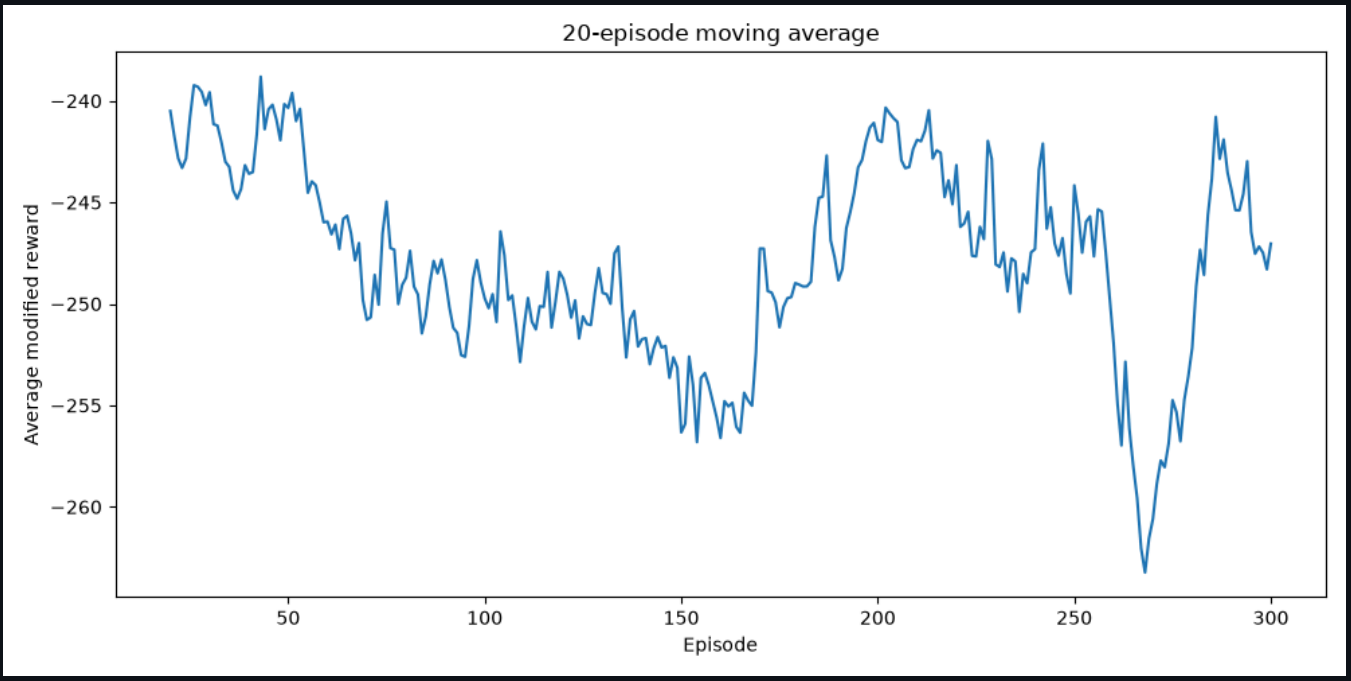
Запуск после исправлений

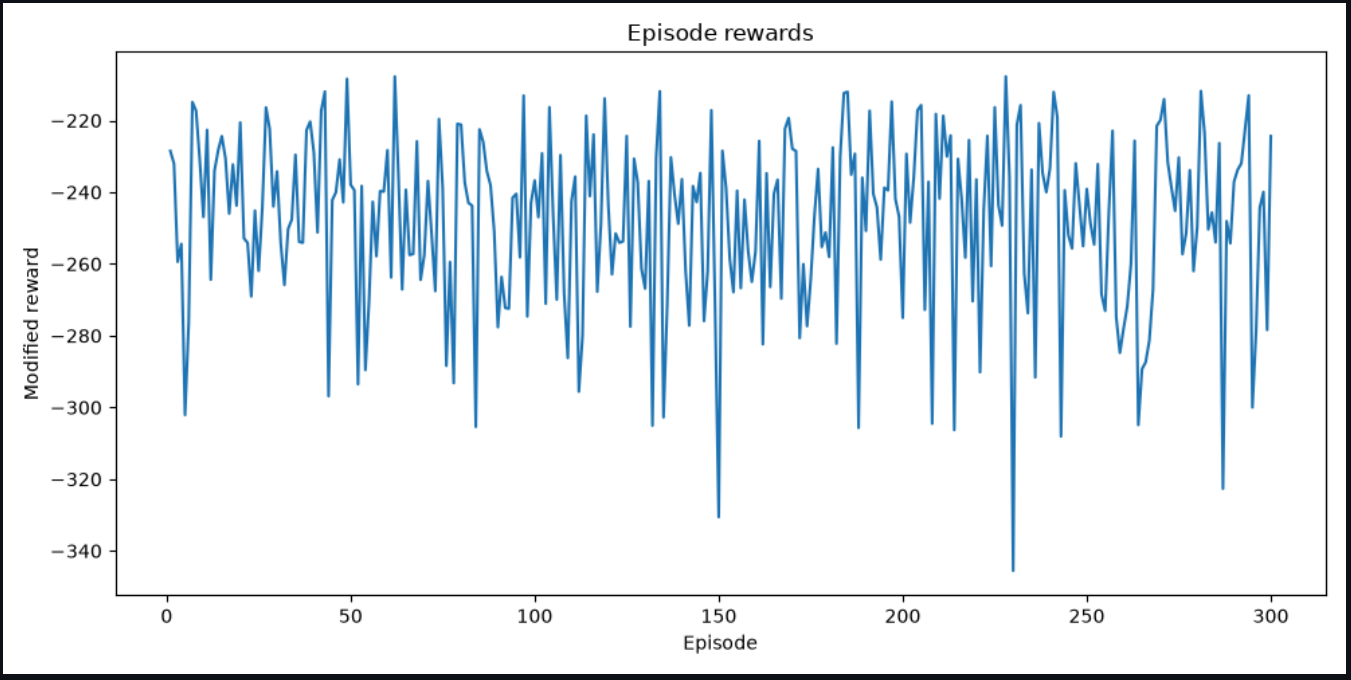
```
❮ python -u main.py --mode train | ts -s
00:00:13 Episode   25 | Reward  -261.92 | Epsilon 0.0100
00:00:22 Episode   50 | Reward  -237.92 | Epsilon 0.0100
00:00:32 Episode   75 | Reward  -238.78 | Epsilon 0.0100
00:00:43 Episode  100 | Reward  -236.74 | Epsilon 0.0100
00:00:55 Episode  125 | Reward  -224.34 | Epsilon 0.0100
00:01:07 Episode  150 | Reward  -330.57 | Epsilon 0.0100
00:01:18 Episode  175 | Reward  -264.33 | Epsilon 0.0100
00:01:30 Episode  200 | Reward  -275.04 | Epsilon 0.0100
00:01:42 Episode  225 | Reward  -216.33 | Epsilon 0.0100
00:01:54 Episode  250 | Reward  -239.10 | Epsilon 0.0100
00:02:07 Episode  275 | Reward  -230.33 | Epsilon 0.0100
00:02:20 Episode  300 | Reward  -224.37 | Epsilon 0.0100
```

```
❯ python -u main.py --mode evaluate
{'mean_reward': -200.0, 'std_reward': 0.0, 'min_reward': -200.0,
'max_reward': -200.0, 'successes': 0, 'episodes': 20}
```


Epsilon
Epsilon
0.035
0.030
0.025
0.020
0.015
0.010
0
50
100
150
200
250
300
Episode


Training loss
Loss
0.06
0.05
0.04
0.03
0.02
0.01
0.00
0
50
100
150
200
250
300
Episode





Все результаты сохранены в папке `results/results_after-fix/`.

Выводы

- Теперь loss падает. Уже на 25 эпизоде лосс упал до 1e-5 и колеблется от 1e-8 до 1e-5. Маленький loss не обязательно означает хорошую модель и по наградам это видно.
- Награды колеблются от -220 до -340. Награда все еще не поднимается.
- По видео видно, что машинка научилась выбирать действие направо, но ей не хватает мощности, чтобы преодолеть холм. Если исправить epsilon, агент

будет больше исследовать, и возможно поймет, что если он заедет на левый холм, ему хватит скорости, чтобы разогнаться и заехать на правый.

Задача 2

Запуск поиска

Я написал два скрипта для поиска гиперпараметров. hp_search.py, hp_search_results.py.

hp_search.py берет конфигурации из hp_search_config.yaml; запускает обучение, валидацию, рендеринг; сохраняет результаты в `results/hp/{name}/`.

Поиск:

```
❱ python -u hp_search.py --config-path hp_search_config.yaml | ts -s |
tee -a hp_search.log
```

Гиперпараметры:

```yaml
base:
  name: base
  seed: 42
  gamma: 0.99
  learning_rate: 1e-3
  batch_size: 64
  buffer_capacity: 100000
  epsilon_start: 1.0
  epsilon_end: 0.05
  epsilon_decay: 0.9999
  target_update_freq: 500
  episodes: 300
```

Параметры изменялись в отдельности.

```yaml
gamma: [0.95, 0.98, 0.99, 0.995, 0.999]
learning_rate: [1e-4, 3e-4, 1e-3, 3e-3, 1e-2]
batch_size: [16, 32, 64, 128, 256, 512]
buffer_capacity: [10_000, 100_000]
epsilon:
    - {start: 0.10, end: 0.01, decay: 0.995}
    - {start: 0.10, end: 0.05, decay: 0.9999}
    - {start: 1.00, end: 0.05, decay: 0.9995}
    - {start: 1.00, end: 0.05, decay: 0.99995}
    - {start: 1.00, end: 0.20, decay: 0.9999}
target_update_freq: [1, 100, 500, 1000, 2000]
episodes: [300, 500]
```

Результаты

```
❱ python hp_search_results.py --root results/hp_results
           name  succ  delta    mean   std  min_reward  max_reward
 eps_decay_9995 20/20 +94.50 -105.50 18.57     -163.00      -84.00
   eps_start_01 20/20 +91.65 -108.35 21.27     -149.00      -84.00
   eps_original 20/20 +75.70 -124.30 27.93     -175.00      -83.00
        tuf_100 20/20 +75.65 -124.35 27.57     -158.00      -83.00
     batch_1024 20/20 +73.85 -126.15 15.00     -144.00     -101.00
      batch_512 19/20 +79.60 -120.40 18.49     -200.00     -112.00
          tuf_1 19/20 +74.05 -125.95 36.26     -200.00      -85.00
      batch_256 17/20 +62.40 -137.60 41.08     -200.00      -84.00
         lr_1e4 16/20 +47.15 -152.85 24.53     -200.00     -136.00
         lr_1e2 11/20 +48.75 -151.25 50.47     -200.00      -89.00
eps_decay_99995 10/20 +31.75 -168.25 31.84     -200.00     -131.00
     gamma_0995 10/20 +22.40 -177.60 32.49     -200.00     -102.00
     buffer_10k  8/20 +24.95 -175.05 30.59     -200.00     -135.00
         lr_3e3  8/20 +11.50 -188.50 16.02     -200.00     -147.00
      gamma_098  7/20 +38.35 -161.65 52.32     -200.00      -85.00
      batch_128  7/20 +30.75 -169.25 41.91     -200.00     -111.00
     gamma_0999  4/20 +22.15 -177.85 44.35     -200.00      -84.00
         lr_3e4  4/20  +3.80 -196.20  7.60     -200.00     -181.00
           base  0/20  +0.00 -200.00  0.00     -200.00     -200.00
    eps_end_020  0/20  +0.00 -200.00  0.00     -200.00     -200.00
       tuf_1000  0/20  +0.00 -200.00  0.00     -200.00     -200.00
       tuf_2000  0/20  +0.00 -200.00  0.00     -200.00     -200.00
      gamma_095  0/20  +0.00 -200.00  0.00     -200.00     -200.00
   episodes_500  0/20  +0.00 -200.00  0.00     -200.00     -200.00
       batch_16  0/20  +0.00 -200.00  0.00     -200.00     -200.00
       batch_32  0/20  +0.00 -200.00  0.00     -200.00     -200.00
```

```
Best: eps_decay_9995
{
  "name": "eps_decay_9995",
  "seed": 42,
  "gamma": 0.99,
  "learning_rate": 0.001,
  "batch_size": 64,
  "buffer_capacity": 100000,
  "epsilon_start": 1.0,
  "epsilon_end": 0.05,
  "epsilon_decay": 0.9995,
  "target_update_freq": 500,
  "episodes": 300
}
{
  "mean_reward": -105.5,
  "std_reward": 18.56744384765625,
  "min_reward": -163.0,
  "max_reward": -84.0,
  "successes": 20,
  "episodes": 20
}
```

#	Эксперимент	LR	Gamma	Batch	Buffer	Epsilon	TUF	Результат
1	base	1e-3	0.99	64	100k	1.0→0.05 (0.9999)	500	0/20; −200.00
2	eps_original	1e-3	0.99	64	100k	0.10→0.01 (0.995)	500	20/20; −124.30
3	eps_start_01	1e-3	0.99	64	100k	0.10→0.05 (0.9999)	500	20/20; −108.35
4	**eps_decay_9995**	1e-3	0.99	64	100k	1.0→0.05 (**0.9995**)	500	**20/20; −105.50**
5	eps_decay_99995	1e-3	0.99	64	100k	1.0→0.05 (0.99995)	500	10/20; −168.25
6	eps_end_020	1e-3	0.99	64	100k	1.0→0.20 (0.9999)	500	0/20; −200.00
7	lr_1e4	1e-4	0.99	64	100k	1.0→0.05 (0.9999)	500	16/20; −152.85
8	lr_3e4	3e-4	0.99	64	100k	1.0→0.05 (0.9999)	500	4/20; −196.20

#	Эксперимент	LR	Gamma	Batch	Buffer	Epsilon	TUF	Результат
9	lr_3e3	3e-3	0.99	64	100k	1.0→0.05 (0.9999)	500	8/20; −188.50
10	lr_1e2	1e-2	0.99	64	100k	1.0→0.05 (0.9999)	500	11/20; −151.25
11	tuf_1	1e-3	0.99	64	100k	1.0→0.05 (0.9999)	1	19/20; −125.95
12	**tuf_100**	1e-3	0.99	64	100k	1.0→0.05 (0.9999)	**100**	**20/20; −124.35**
13	tuf_1000	1e-3	0.99	64	100k	1.0→0.05 (0.9999)	1000	0/20; −200.00
14	tuf_2000	1e-3	0.99	64	100k	1.0→0.05 (0.9999)	2000	0/20; −200.00
15	gamma_095	1e-3	0.95	64	100k	1.0→0.05 (0.9999)	500	0/20; −200.00
16	gamma_098	1e-3	0.98	64	100k	1.0→0.05 (0.9999)	500	7/20; −161.65
17	gamma_0995	1e-3	0.995	64	100k	1.0→0.05 (0.9999)	500	10/20; −177.60
18	gamma_0999	1e-3	0.999	64	100k	1.0→0.05 (0.9999)	500	4/20; −177.85
19	buffer_10k	1e-3	0.99	64	10k	1.0→0.05 (0.9999)	500	8/20; −175.05
20	episodes_500	1e-3	0.99	64	100k	1.0→0.05 (0.9999)	500	0/20; −200.00
21	batch_16	1e-3	0.99	16	100k	1.0→0.05 (0.9999)	500	0/20; −200.00
22	batch_32	1e-3	0.99	32	100k	1.0→0.05 (0.9999)	500	0/20; −200.00
23	batch_128	1e-3	0.99	128	100k	1.0→0.05 (0.9999)	500	7/20; −169.25
24	batch_256	1e-3	0.99	256	100k	1.0→0.05 (0.9999)	500	17/20; −137.60
25	batch_512	1e-3	0.99	512	100k	1.0→0.05 (0.9999)	500	19/20; −120.40
26	batch_1024	1e-3	0.99	1024	100k	1.0→0.05 (0.9999)	500	20/20; −126.15

Выводы

- epsilon — влияет сильнее всего. Лучшее: decay=0.9995 (20/20). Плохое: decay=0.9999 (0/20). Важна скорость перехода к эксплуатации.
- target_update_freq — влияет сильно. Лучшее: 100 (20/20).
- batch_size — влияет заметно. При увеличении увеличивается качество. Улучшение от 512 к 1024 уже не так заметно. Лучшее: 1024 (20/20). маленький батч = градиент по шуму.
- learning_rate — влияет заметно. Лучшее: 1e-4 (16/20); 1e-2 (11/20); середина (1e-3) → 0/20.
- gamma — влияет слабо. Лучшее: 0.995 (10/20).
- buffer_capacity — влияет слабо. Лучшее: 10k (8/20) — маленький буфер быстрее вытесняет шум.
- episodes — не влияет. 500 vs 300 одинаковый результат.
- epsilon_end — не влияет. 0.20 vs 0.05 → 0/20.

Решающие конфиги: eps_decay_9995, tuf_100, batch_512, eps_start_01.

Задача 3

Используя результаты из задачи 2, попробуем найти оптимальную комбинацию гиперпараметров.

Конфиг

> hp_search_config_combined.yaml

```yaml
results_root: "results/hp_results_combination"

base: &base
  name: base
  seed: 42
  gamma: 0.99
  learning_rate: 1e-3
  batch_size: 64
  buffer_capacity: 100000
  epsilon_start: 1.0
  epsilon_end: 0.05
  epsilon_decay: 0.9999
```

```yaml
  target_update_freq: 500
  episodes: 300

experiments:
  - <<: *base
    name: base_combination
    gamma: 0.9995
    learning_rate: 1e-4
    batch_size: 512
    buffer_capacity: 10000
    epsilon_decay: 0.9995
    target_update_freq: 100

  - <<: *base
    name: final_a
    epsilon_decay: 0.9995

  - <<: *base
    name: final_b_conservative
    epsilon_decay: 0.9995
    target_update_freq: 100
    batch_size: 256

  - <<: *base
    name: final_c_aggressive
    epsilon_decay: 0.9995
    target_update_freq: 100
    batch_size: 1024

  - <<: *base
    name: final_d_alt_eps
    epsilon_start: 0.1
    epsilon_decay: 0.9999
    target_update_freq: 100
    batch_size: 1024
```

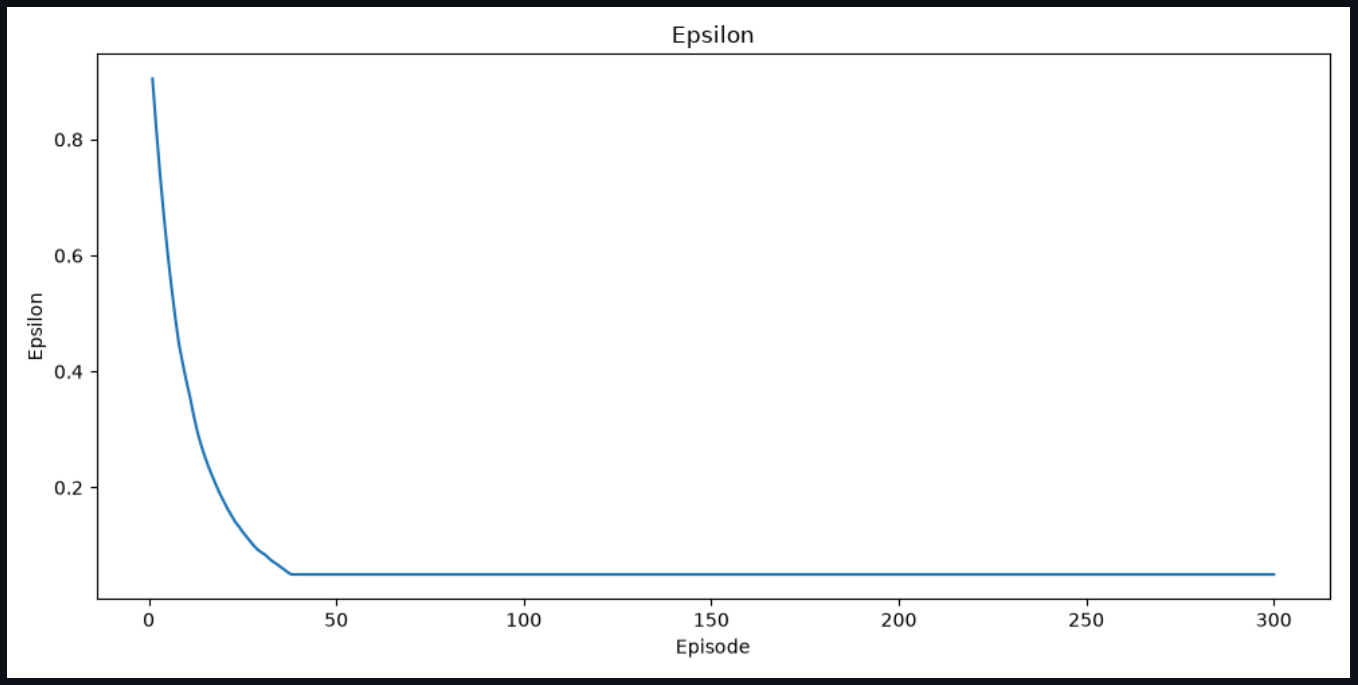
Результаты

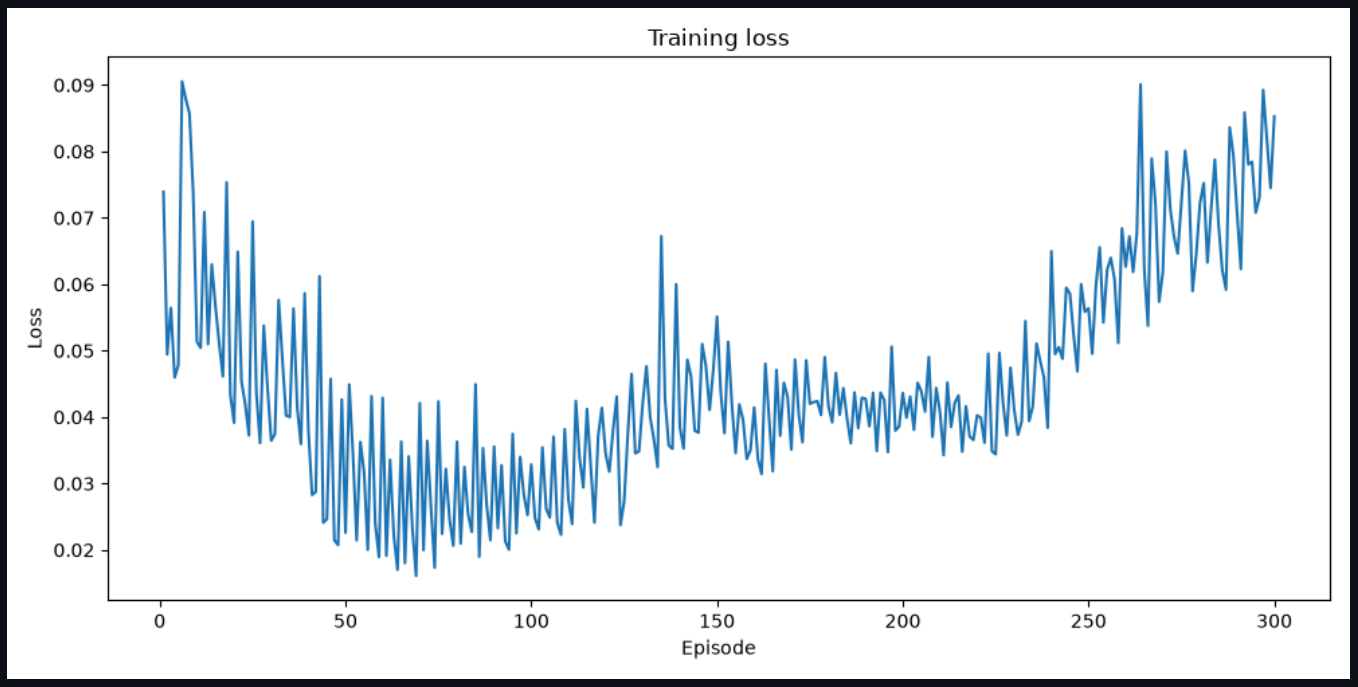
```
❱ python hp_search_results.py --root results/hp_results_combination
                name  succ  delta    mean   std  min_reward  max_reward
             final_a 20/20 +98.00 -102.00 10.96     -114.00      -83.00
     final_d_alt_eps 20/20 +96.55 -103.45 14.48     -155.00      -84.00
final_b_conservative 20/20 +74.15 -125.85 32.11     -184.00      -84.00
  final_c_aggressive 20/20 +72.45 -127.55 27.13     -166.00      -85.00
```

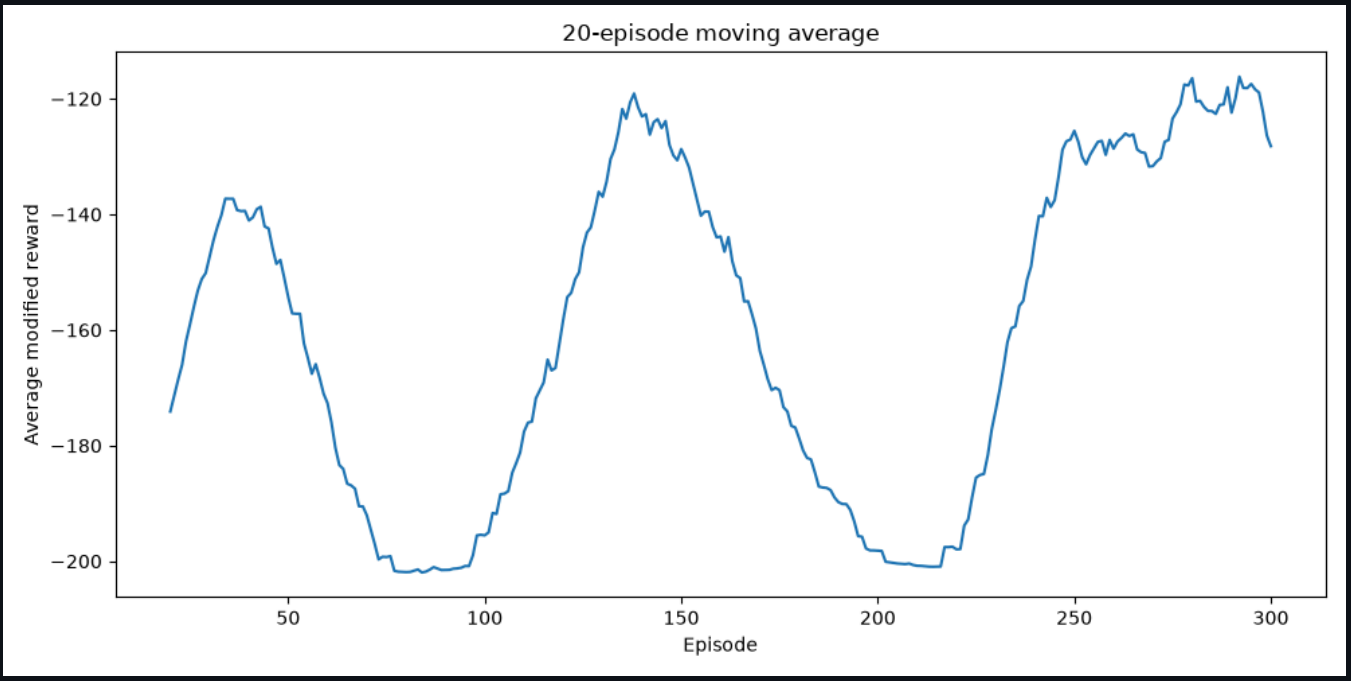
```
       final_a_lr_1e4 12/20 +55.65 -144.35 47.53     -200.00      -86.00
    base_combination  0/20  +0.00 -200.00  0.00     -200.00     -200.00
                base  0/20  +0.00 -200.00  0.00     -200.00     -200.00

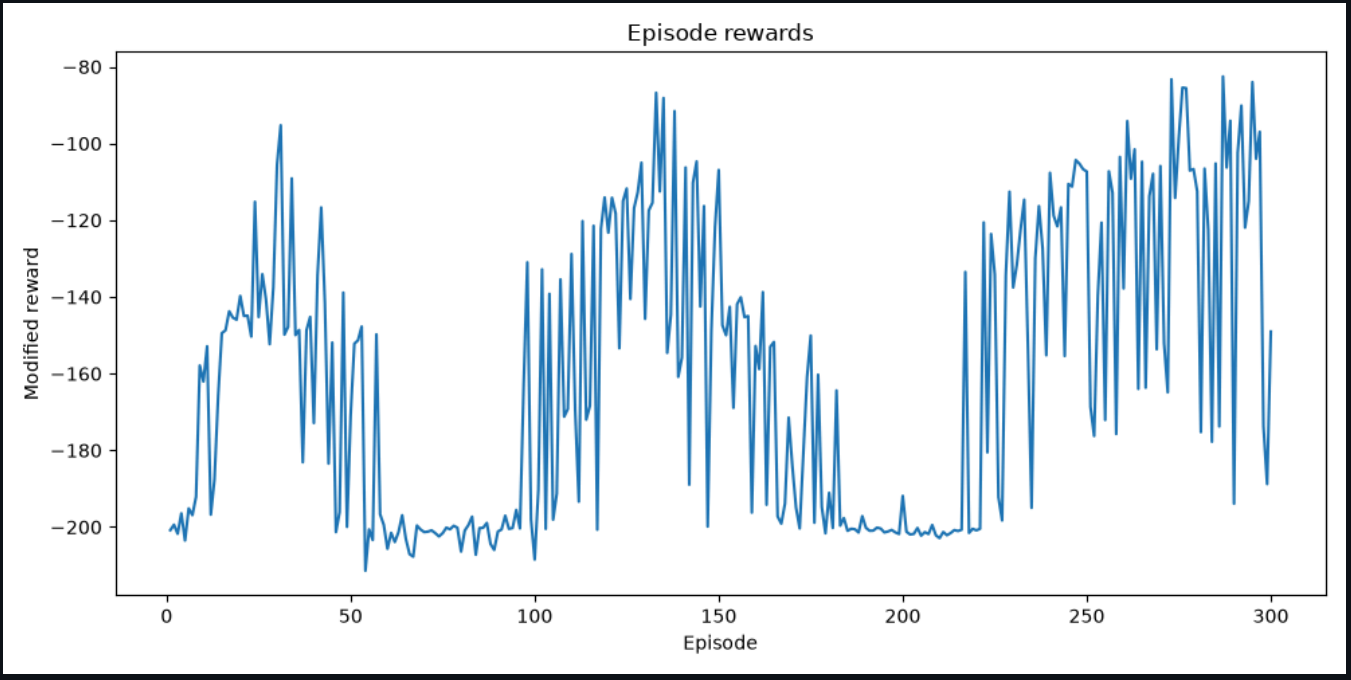
Best: final_a
{
  "name": "final_a",
  "seed": 42,
  "gamma": 0.99,
  "learning_rate": 0.001,
  "batch_size": 64,
  "buffer_capacity": 100000,
  "epsilon_start": 1.0,
  "epsilon_end": 0.05,
  "epsilon_decay": 0.9995,
  "target_update_freq": 500,
  "episodes": 300
}
{
  "mean_reward": -102.0,
  "std_reward": 10.963576316833496,
  "min_reward": -114.0,
  "max_reward": -83.0,
  "successes": 20,
  "episodes": 20
}
```

#	Эксперимент	LR	Gamma	Batch	Buffer	Epsilon	TUF	Результат
27	base_combination	1e-4	0.9995	512	10k	1.0→0.05 (0.9995)	100	0/20; −200.00
28	**final_a**	1e-3	0.99	64	100k	1.0→0.05 (**0.9995**)	500	**20/20; −102.00**
29	final_b_conservative	1e-3	0.99	256	100k	1.0→0.05 (0.9995)	100	20/20; −125.85
30	final_c_aggressive	1e-3	0.99	1024	100k	1.0→0.05 (0.9995)	100	20/20; −127.55
31	final_d_alt_eps	1e-3	0.99	1024	100k	0.1→0.05 (0.9999)	100	20/20; −103.45
32	final_a_lr_1e4	1e-4	0.99	64	100k	1.0→0.05 (0.9995)	500	12/20; −144.35


Epsilon
Epsilon
0.8
0.6
0.4
0.2
0
50
100
150
200
250
300
Episode


Training loss
Loss
0.09
0.08
0.07
0.06
0.05
0.04
0.03
0.02
0
50
100
150
200
250
300
Episode





Выводы

- наивное исправление всех параметров привело к плохому результату.
- лучший результат показал конфиг final_a с простым изменением epsilon_decay на 0.9995.
- по видео видно, что решение получилось оптимальное. машинка заезжает на левый холм, разгоняется вправо и заезжает на правый холм с оптимальной скоростью. (видео можно найти в `results/hp_results_combination/final_a/videos/evaluation_episode.mp4`).

Новый main.py

Конфиг

Финальный конфиг (results/main/config.json):

```
{
  "name": "final_a",
  "seed": 42,
  "gamma": 0.99,
  "learning_rate": 0.001,
  "batch_size": 64,
  "buffer_capacity": 100000,
  "epsilon_start": 1.0,
  "epsilon_end": 0.05,
  "epsilon_decay": 0.9995,
  "target_update_freq": 500,
  "train_episodes": 300,
  "val_episodes": 20
}
```

Воспроизводимость

Seed зафиксирован.

```
def set_seed(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
```

Результаты

Сохраним найденные значения гиперпараметров в main.py и запустим.

```
❯ python -u main.py --mode train
Episode   25 | Reward  -149.51 | Epsilon 0.1274
Episode   50 | Reward  -116.80 | Epsilon 0.0500
Episode   75 | Reward  -200.81 | Epsilon 0.0500
Episode  100 | Reward  -200.66 | Epsilon 0.0500
Episode  125 | Reward  -121.19 | Epsilon 0.0500
Episode  150 | Reward   -98.42 | Epsilon 0.0500
Episode  175 | Reward  -205.00 | Epsilon 0.0500
Episode  200 | Reward  -201.69 | Epsilon 0.0500
Episode  225 | Reward  -111.86 | Epsilon 0.0500
Episode  250 | Reward  -108.84 | Epsilon 0.0500
Episode  275 | Reward   -84.76 | Epsilon 0.0500
Episode  300 | Reward  -178.04 | Epsilon 0.0500
```

```
❯ python main.py --mode evaluate
{
  "mean_reward": -112.25,
  "std_reward": 32.466712951660156,
  "min_reward": -200.0,
  "max_reward": -84.0,
  "successes": 19,
  "episodes": 20
}

❯ python -u main.py --mode render
IMAGEIO FFMPEG_WRITER WARNING: input image is not divisible by
macro_block_size=16, resizing from (600, 400) to (608, 400) to ensure
video compatibility with most codecs and players. To prevent resizing,
make your input imag
e divisible by the macro_block_size or set the macro_block_size to 1
(risking incompatibility).
Saved video to results/main/videos/evaluation_episode.mp4
```

Выводы по ДЗ

Основные ошибки:

1. Самая важная ошибка - баг с target. В коде есть ошибка с подсчетом targets: 107 строчка: `targets = rewards + self.gamma * next_q * dones`. Нужно `targets = rewards + self.gamma * next_q * (1 - dones)`.
2. Не используется `target_update_freq`. target_net обновляется каждый шаг, что

неправильно. Этот баг неявно привел к бесполезности target_net. DQN стал наивнее.

3. Мы не используем truncated, которое возвращает env.step.

Главные гиперпараметры: